

Bendy 记账 (bendy-jizhang) — 完整产品设计文档

版本：v2.0 | 日期：2026-05-28 框架：Flutter | 平台：Android + Desktop + Web 参考：ezBookkeeping Android v0.9.1 | 精简：仅保留记账+分析核心

1. 项目概述

1.1 定位

Bendy 记账是一款纯本地、轻量级、跨平台记账应用，聚焦「记账」与「分析」两大核心能力。一套 Dart 代码，同时打包 Android APK、桌面（Windows/macOS/Linux）和 Web。所有数据存于本地 SQLite，无服务器、无注册、无登录。

1.2 核心原则

原则	说明
跨平台	Flutter 一套代码 → Android + Desktop + Web 三端发布
纯本地	SQLite 本地存储，无服务器、无注册、无登录
极简	只做记账+分析，砍掉标签/模板/汇率/导入/对账/云端等非核心
快	打开即记，首页展示多周期汇总，零等待
可分析	按分类/时间/账户多维度统计，饼图+柱状图+趋势线
自适应	移动端底部导航，桌面端侧边导航栏，Web 响应式

1.3 技术栈

层	技术	包名	版本
框架	Flutter + Dart	—	Flutter 3.27 / Dart 3.6
UI	Material 3	flutter	SDK 内置
状态管理	Riverpod	flutter_riverpod	^2.6
数据库	drift (SQLite ORM)	drift + drift_flutter	^2.22

层	技术	包名	版本
导航	go_router	go_router	^14.6
本地存储	shared_preferences	shared_preferences	^2.3
图表	fl_chart	fl_chart	^0.69
i18n	intl + flutter_localizations	—	SDK 内置
测试	flutter_test + mockito	—	SDK 内置
桌面	sqlite3 (native)	sqlite3_flutter_libs	^0.5
Web	sqlite3 (WASM)	drift/web	drift 内置

1.4 三端差异处理

维度	Android	Desktop	Web
SQLite 引擎	sqlite3_flutter_libs (native)	sqlite3_flutter_libs (native)	drift WASM (IndexedDB 模拟)
导航模式	底部 Tab	侧边 Rail / Drawer	侧边 Rail
窗口	全屏	自由调整 800×600~	浏览器窗口
文件路径	getApplicationDocumentsDirectory	当前目录	浏览器存储
快捷键	无	Ctrl+N 新建, Ctrl+S 保存	Ctrl+N, Ctrl+S
入口	main.dart → runApp	main.dart → configureWindowTitle	main.dart → web 同 desktop

2. 功能清单

2.1 保留功能（参考 [ezBookkeeping](#)）

#	功能	原项目对应	说明
1	首页仪表盘	HomeScreen	今日/本周/本月/本年 收支汇总 + 最近交易
2		TransactionEditScreen	

#	功能	原项目对应	说明
	记账（支出/收入/转账）		三种类型，选分类/账户/日期/备注
3	交易列表	TransactionListScreen	按月分组，支持搜索
4	删除/编辑交易	TransactionEditScreen(isEdit)	长按或点击进入编辑
5	账户管理	AccountListScreen + AccountEditScreen	资产/负债两类，增删改
6	分类管理	CategoryListScreen + CategoryEditScreen	支出/收入两类，支持二级分类
7	统计分析	StatisticsScreen	饼图/柱状图/趋势线
8	设置	SettingsScreen	主题色、暗色模式、货币
9	纯本地启动	LoginScreen (standalone)	无登录流程，首次打开直接进入首页

2.2 砍掉功能

原功能	砍掉原因
标签/标签组	非核心，分类已够用
模板	复杂度偏高
汇率/多币种	简化为单币种
导入/导出	非核心
对账	非核心
洞察分析器	过于复杂
定时交易	非核心
应用锁/生物识别	首版不做
2FA/TOTP	纯本地无需
云端同步	纯本地无服务器
用户系统	纯本地无需
图片/附件	非核心
AI/语音	非核心

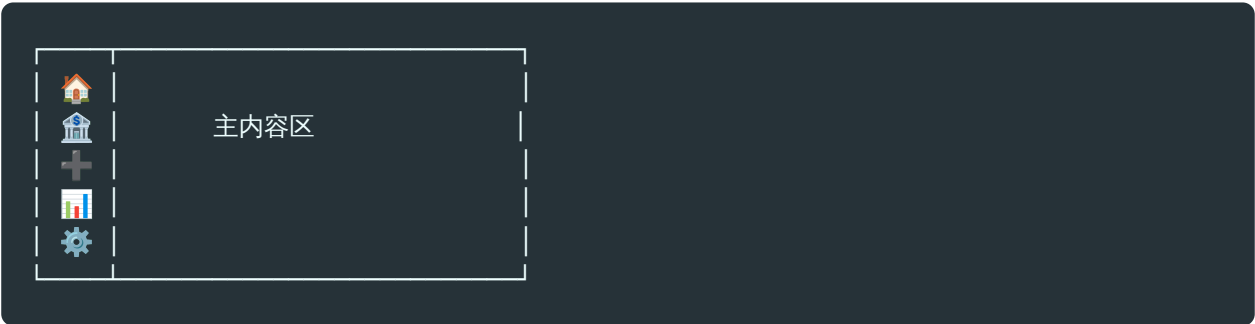
3. 页面结构与导航

3.1 导航架构

移动端 (窄屏 < 600dp) : 底部 Tab 导航



桌面/Web (宽屏 >= 600dp) : 侧边 NavigationRail



3.2 路由表 (go_router)

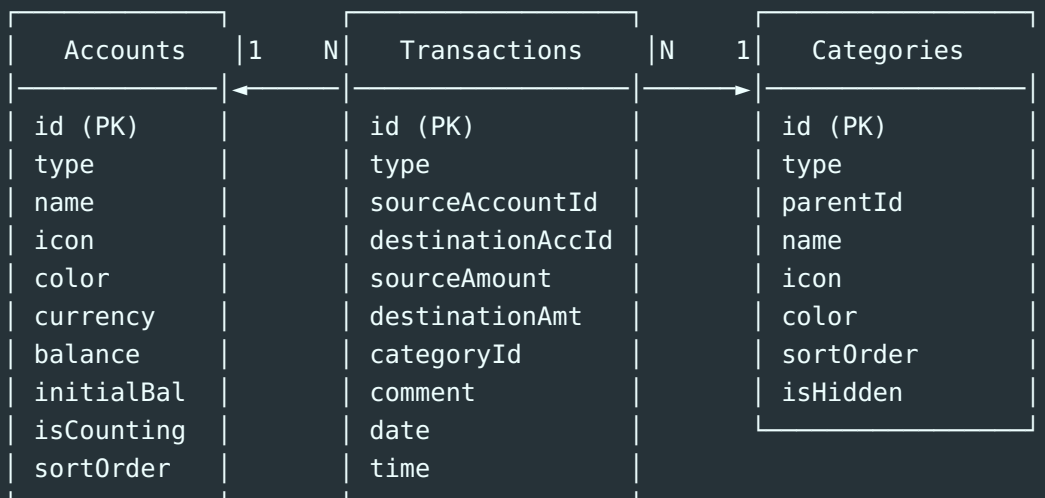
路由	页面	说明
/	HomeScreen	首页仪表盘
/accounts	AccountListScreen	账户列表
/accounts/add	AccountEditScreen	新建账户
/accounts/:id	AccountEditScreen	编辑账户
/transactions/add	TransactionEditScreen	新建交易
/transactions/:id	TransactionEditScreen	编辑交易
/transactions	TransactionListScreen	交易列表
/categories	CategoryListScreen	分类管理
/categories/add	CategoryEditScreen	新建分类
/categories/:id	CategoryEditScreen	编辑分类
/statistics	StatisticsScreen	统计分析
/settings	SettingsScreen	设置

3.3 Shell 路由结构

```
GoRouter(  
  routes: [  
    ShellRoute(  
      builder: (context, state, child) => AppScaffold(child: child),  
      routes: [  
        GoRoute(path: '/', builder: (_, __) => HomeScreen()),  
        GoRoute(path: '/accounts', builder: (_, __) => AccountListScreen()),  
        GoRoute(path: '/transactions', builder: (_, __) => TransactionListScreen()),  
        GoRoute(path: '/statistics', builder: (_, __) => StatisticsScreen()),  
        GoRoute(path: '/settings', builder: (_, __) => SettingsScreen()),  
      ],  
    ),  
    GoRoute(path: '/accounts/add', builder: (_, __) => AccountEditScreen()),  
    GoRoute(path: '/accounts/:id', builder: (_, s) => AccountEditScreen(id: int.parse(s.pathParameters['id'])),  
    GoRoute(path: '/transactions/add', builder: (_, __) => TransactionEditScreen()),  
    GoRoute(path: '/transactions/:id', builder: (_, s) => TransactionEditScreen(id: int.parse(s.pathParameters['id'])),  
    GoRoute(path: '/categories', builder: (_, __) => CategoryListScreen()),  
    GoRoute(path: '/categories/add', builder: (_, __) => CategoryEditScreen()),  
    GoRoute(path: '/categories/:id', builder: (_, s) => CategoryEditScreen(id: int.parse(s.pathParameters['id'])),  
  ],  
);
```

4. 数据模型 (drift)

4.1 实体关系图



4.2 Accounts 表

```
class Accounts extends Table {
  IntColumn get id => integer().autoIncrement();
  TextColumn get type => textEnum<AccountType>(); // ASSET, LIABILITY
  TextColumn get name => text().withLength(min: 1, max: 50());
  TextColumn get icon => text().withLength(min: 1, max: 50());
  TextColumn get color => text().withLength(min: 7, max: 7()); // "#FF5722"
  TextColumn get currency => text().withLength(min: 3, max: 3()); // "CNY"
  RealColumn get balance => real().withDefault(const Constant(0.0));
  RealColumn get initialBalance => real().withDefault(const Constant(0.0));
  BoolColumn get isCounting => boolean().withDefault(const Constant(true));
  IntColumn get sortOrder => integer().withDefault(const Constant(0));
}
```

4.3 Transactions 表

```
class Transactions extends Table {
  IntColumn get id => integer().autoIncrement();
  TextColumn get type => textEnum<TransactionType>(); // EXPENSE, INCOME, TRANSFER
  IntColumn get sourceAccountId => integer().references(Accounts, #id());
  IntColumn get destinationAccountId => integer().nullable().references(Accounts, #id());
  RealColumn get sourceAmount => real();
  RealColumn get destinationAmount => real().nullable();
  IntColumn get categoryId => integer().nullable().references(Categories, #id());
  TextColumn get comment => text().nullable().withLength(max: 200());
  TextColumn get date => text(); // "2026-05-28"
  TextColumn get time => text().nullable(); // "14:30"
}
```

4.4 Categories 表

```
class Categories extends Table {
  IntColumn get id => integer().autoIncrement();
  TextColumn get type => textEnum<CategoryType>(); // EXPENSE, INCOME
  IntColumn get parentId => integer().nullable().references(Categories, #id());
  TextColumn get name => text().withLength(min: 1, max: 30());
  TextColumn get icon => text().withLength(min: 1, max: 50());
  TextColumn get color => text().withLength(min: 7, max: 7());
  IntColumn get sortOrder => integer().withDefault(const Constant(0));
  BoolColumn get isHidden => boolean().withDefault(const Constant(false));
}
```

4.5 枚举

```
enum AccountType { asset, liability }
enum TransactionType { expense, income, transfer }
enum CategoryType { expense, income }
```

4.6 与原项目对比

实体	原项目 (Room/Kotlin)	Bendy (drift/Dart)	变化
Account	ezbk_accounts (14字段+userId)	accounts (10字段)	砍userId, drift自动生成PK
Transaction	ezbk_transactions (12字段+userId+tagIds+...)	transactions (10字段)	砍userId/tagIds/templateId/经纬度
Category	ezbk_categories (9字段+userId)	categories (8字段)	砍userId
其余11表	存在	✗ 全部砍掉	纯本地精简

5. DAO 层 (drift Accessor)

5.1 AccountDao

```
@DriftAccessor(tables: [Accounts])
class AccountDao extends DatabaseAccessor<AppDatabase> with _$AccountDaoMixin {
    AccountDao(AppDatabase db) : super(db);

    Stream<List<Account>> watchAll() =>
        (select(accounts)..orderBy([(t) => OrderingTerm.asc(t.sortOrder)]))..watch();

    Stream<Account?> watchById(int id) =>
        (select(accounts)..where((t) => t.id.equals(id)))..watchSingleOrNull();

    Stream<List<Account>> watchByType(AccountType type) =>
        (select(accounts)..where((t) => t.type.equalsValue(type))
            ..orderBy([(t) => OrderingTerm.asc(t.sortOrder)]))..watch();

    Future insertOrUpdate(AccountsCompanion entry) => into(accounts).insertOnConflictUpdate(entry);
    Future deleteEntry(Account entry) => delete(accounts).delete(entry);
}
```

5.2 TransactionDao

```
@DriftAccessor(tables: [Transactions])
class TransactionDao extends DatabaseAccessor<AppDatabase> with _$TransactionDaoMixin {
    TransactionDao(AppDatabase db) : super(db);

    Stream<List<Transaction>> watchByDateRange(String start, String end) =>
        (select(transactions)..where((t) => t.date.isBetweenValues(start, end))
            ..orderBy([(t) => OrderingTerm.desc(t.date), (t) => OrderingTerm.desc(t.time)]))
            .watch();

    Stream<Transaction?> watchById(int id) =>
        (select(transactions)..where((t) => t.id.equals(id))).watchSingleOrNull();

    Stream<List<Transaction>> watchByCategory(int catId) =>
        (select(transactions)..where((t) => t.categoryId.equals(catId))).watch();

    Stream<List<Transaction>> watchByAccount(int accId) =>
        (select(transactions)..where((t) => t.sourceAccountId.equals(accId) | t.destinationAccountId.equals(accId))).watch();

    Future insertOrUpdate(TransactionsCompanion entry) => into(transactions).insertOnConflictUpdate(entry);
    Future deleteEntry(Transaction entry) => delete(transactions).delete(entry);
}
```

5.3 CategoryDao

```
@DriftAccessor(tables: [Categories])
class CategoryDao extends DatabaseAccessor<AppDatabase> with _$CategoryDaoMixin {
    CategoryDao(AppDatabase db) : super(db);

    Stream<List<Category>> watchByType(CategoryType type) =>
        (select(categories)..where((t) => t.type.equalsValue(type))
            ..orderBy([(t) => OrderingTerm.asc(t.sortOrder)]))
            .watch();

    Stream<List<Category>> watchAll() =>
        (select(categories)..orderBy([(t) => OrderingTerm.asc(t.sortOrder)]))
            .watch();

    Stream<Category?> watchById(int id) =>
        (select(categories)..where((t) => t.id.equals(id))).watchSingleOrNull();

    Future insertOrUpdate(CategoriesCompanion entry) => into(categories).insertOnConflictUpdate(entry);
    Future deleteEntry(Category entry) => delete(categories).delete(entry);
}
```

6. Repository 层

```
class AccountRepository {
    final AccountDao _dao;
    AccountRepository(this._dao);

    Stream<List<Account>> watchAll() => _dao.watchAll();
    Stream<Account?> watchById(int id) => _dao.watchById(id);
    Stream<List<Account>> watchByType(AccountType type) => _dao.watchByType(type);
    Future save(AccountsCompanion entry) => _dao.insertOrUpdate(entry);
    Future delete(Account entry) => _dao.deleteEntry(entry);
}

class TransactionRepository {
    final TransactionDao _dao;
    TransactionRepository(this._dao);

    Stream<List<Transaction>> watchByDateRange(String start, String end) => _dao.watchByDateRange(start, end);
    Stream<Transaction?> watchById(int id) => _dao.watchById(id);
    Future save(TransactionsCompanion entry) => _dao.insertOrUpdate(entry);
    Future delete(Transaction entry) => _dao.deleteEntry(entry);
}

class CategoryRepository {
    final CategoryDao _dao;
    CategoryRepository(this._dao);

    Stream<List<Category>> watchByType(CategoryType type) => _dao.watchByType(type);
    Stream<List<Category>> watchAll() => _dao.watchAll();
    Stream<Category?> watchById(int id) => _dao.watchById(id);
    Future save(CategoriesCompanion entry) => _dao.insertOrUpdate(entry);
    Future delete(Category entry) => _dao.deleteEntry(entry);
}
```

7. Riverpod Provider 设计

7.1 数据库 Provider

```
@Riverpod(keepAlive: true)
AppDatabase appDatabase(Ref ref) => AppDatabase();

@Riverpod(keepAlive: true)
AccountDao accountDao(Ref ref) => ref.watch(appDatabaseProvider).accountDao;

@Riverpod(keepAlive: true)
```

```
TransactionDao transactionDao(Ref ref) => ref.watch(appDatabaseProvider).transactionDao;

@Riverpod(keepAlive: true)
CategoryDao categoryDao(Ref ref) => ref.watch(appDatabaseProvider).categoryDao;
```

7.2 Repository Provider

```
@Riverpod(keepAlive: true)
AccountRepository accountRepository(Ref ref) =>
    AccountRepository(ref.watch(accountDaoProvider));

@Riverpod(keepAlive: true)
TransactionRepository transactionRepository(Ref ref) =>
    TransactionRepository(ref.watch(transactionDaoProvider));

@Riverpod(keepAlive: true)
CategoryRepository categoryRepository(Ref ref) =>
    CategoryRepository(ref.watch(categoryDaoProvider));
```

7.3 数据 Stream Provider

```
@riverpod
Stream<List<Account>> allAccounts(Ref ref) =>
    ref.watch(accountRepositoryProvider).watchAll();

@riverpod
Stream<List<Account>> accountsByType(Ref ref, AccountType type) =>
    ref.watch(accountRepositoryProvider).watchByType(type);

@riverpod
Stream<List<Transaction>> transactionsByDateRange(Ref ref, String start, String end) =>
    ref.watch(transactionRepositoryProvider).watchByDateRange(start, end);

@riverpod
Stream<List<Category>> categoriesByType(Ref ref, CategoryType type) =>
    ref.watch(categoryRepositoryProvider).watchByType(type);

@riverpod
Stream<List<Category>> allCategories(Ref ref) =>
    ref.watch(categoryRepositoryProvider).watchAll();
```

7.4 设置 Provider

```
@Riverpod(keepAlive: true)
class SettingsNotifier extends _$SettingsNotifier {
```

```

@override
SettingsState build() => SettingsState(themeMode: ThemeMode.system, currency: CNY');

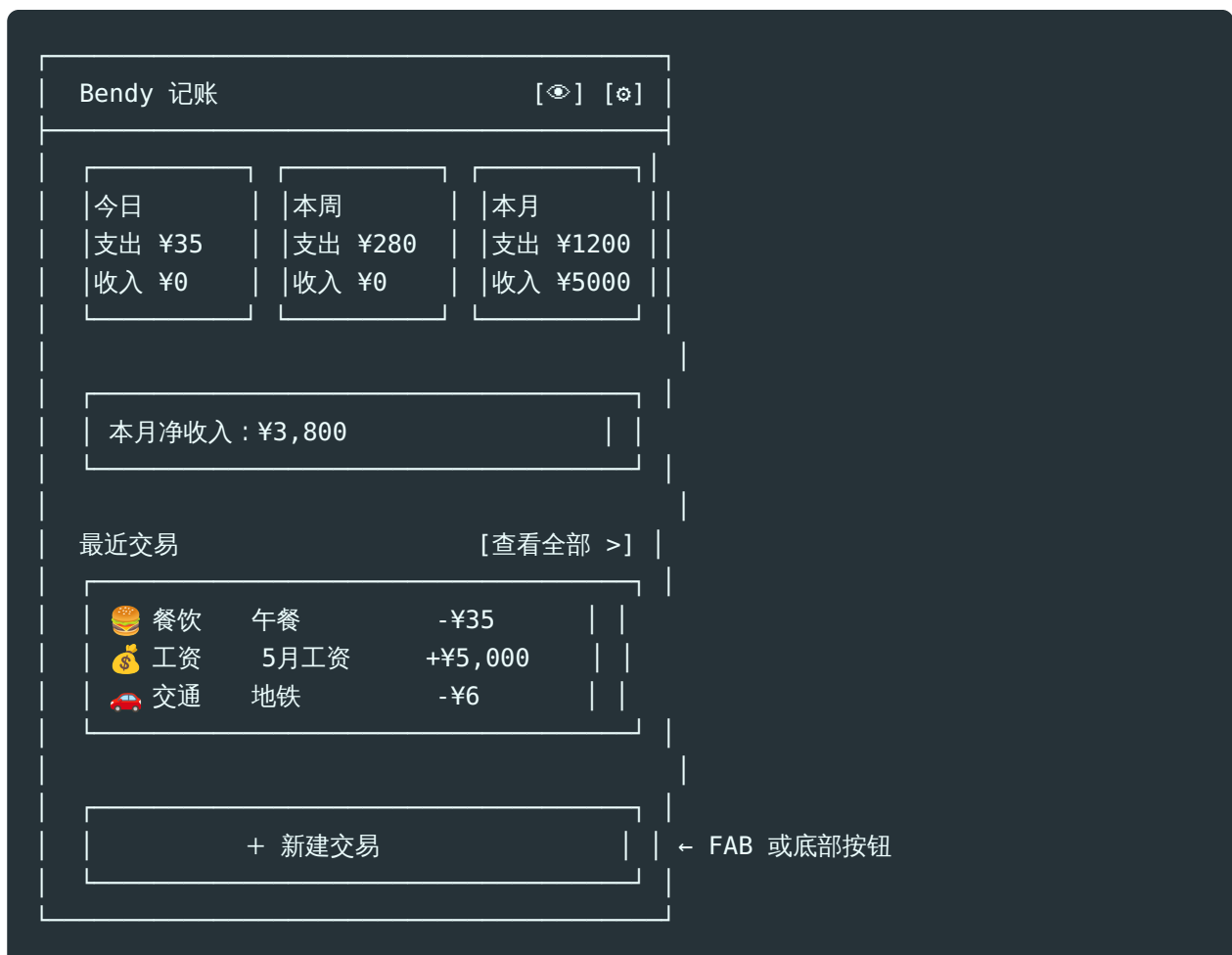
void setThemeMode(ThemeMode mode) => state = state.copyWith(themeMode: mode);
void setCurrency(String cur) => state = state.copyWith(currency: cur);
}

@freezed
class SettingsState with _$SettingsState {
  const factory SettingsState({
    required ThemeMode themeMode,
    required String currency,
    String themeColor = '#6200EE',
    bool showAmount = true,
  }) = _SettingsState;
}

```

8. Screen 设计

8.1 首页仪表盘 (HomeScreen)

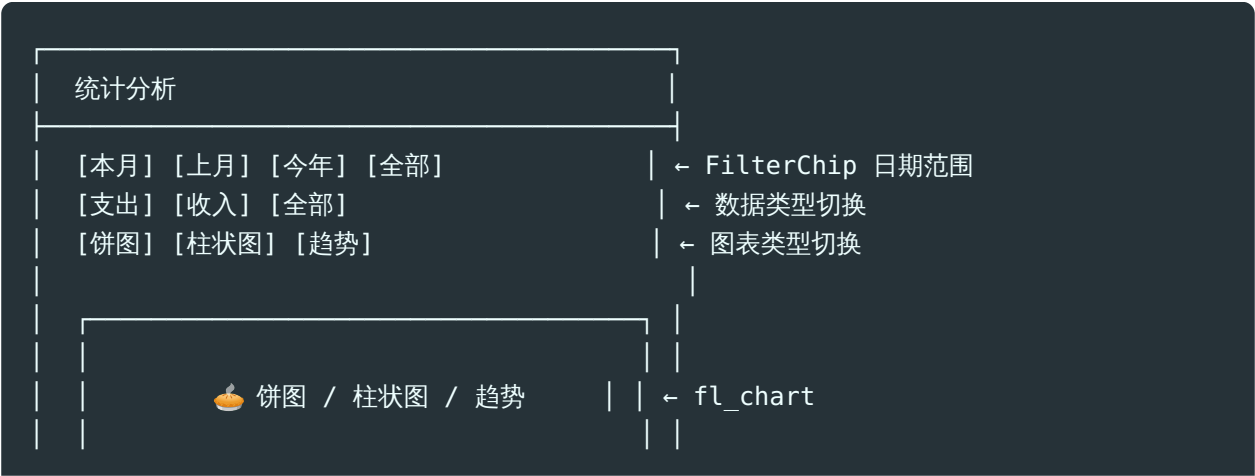


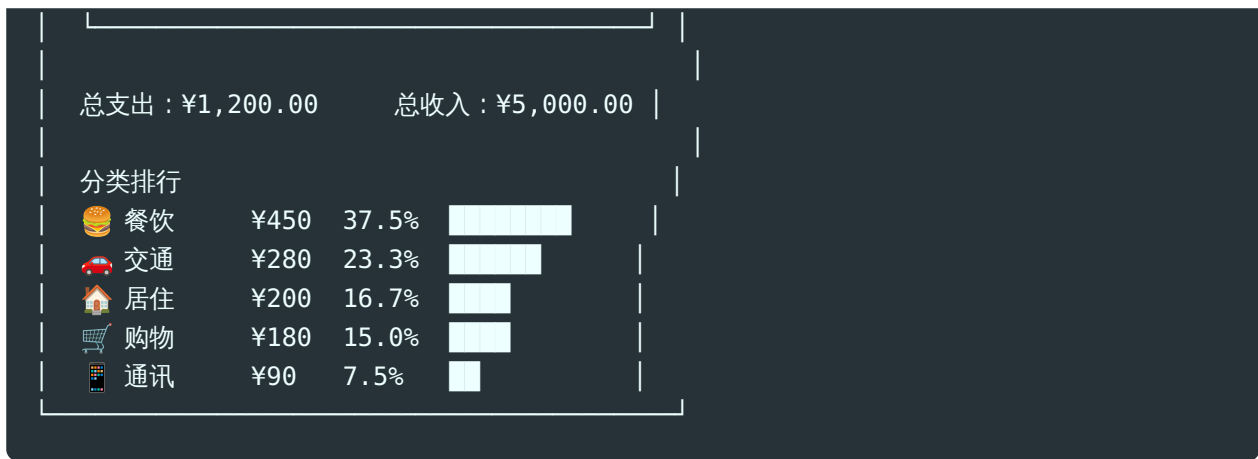
桌面端差异：左侧 NavigationRail，内容区更宽，卡片横排4列（今日/本周/本月/本年）

8.2 记账页 (TransactionEditScreen)



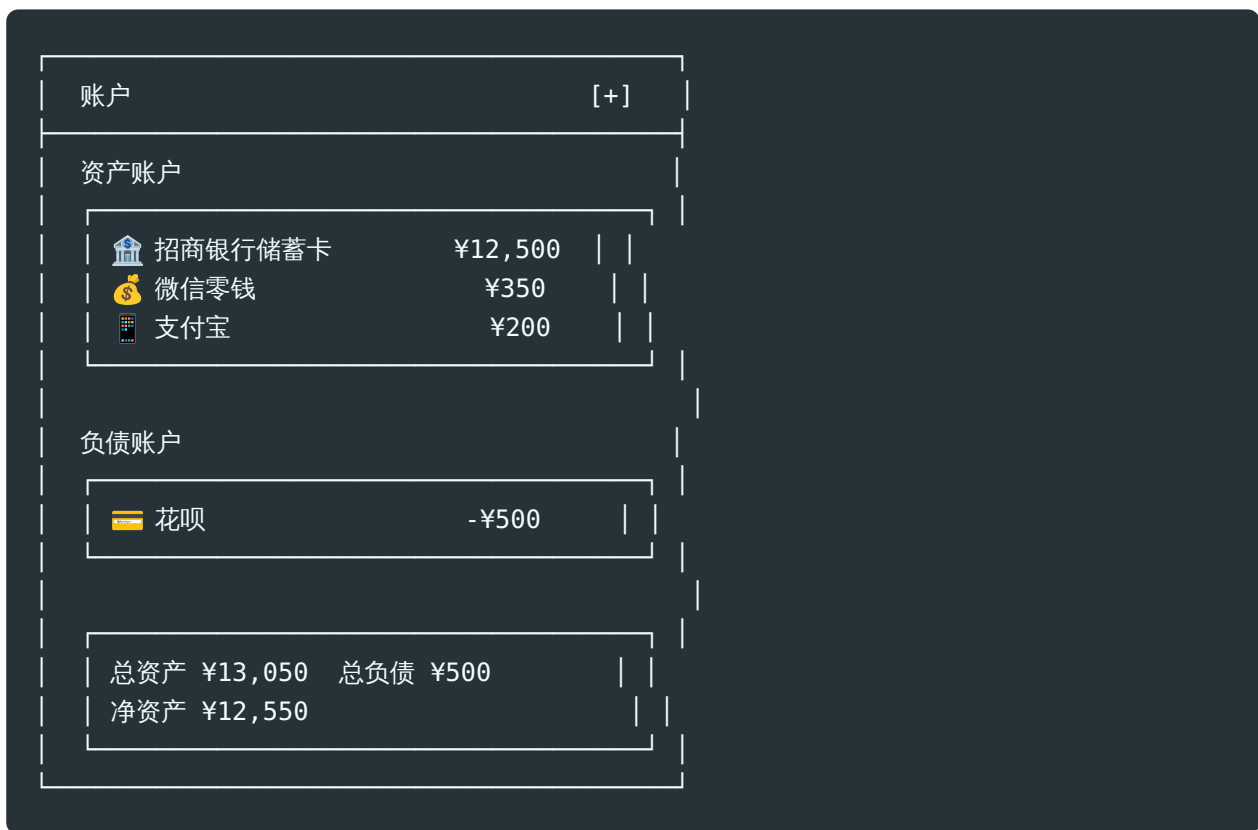
8.3 统计分析 (StatisticsScreen)



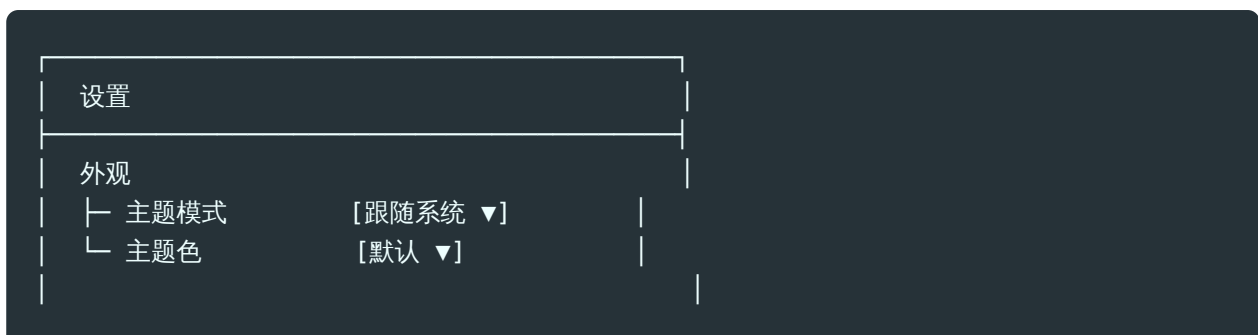


桌面端差异：图表区更宽，支持悬浮 Tooltip，右侧分类排行面板

8.4 账户列表 (AccountListScreen)



8.5 设置 (SettingsScreen)



```
| 数据 |
| └─ 默认货币 [CNY ▼] |
| └─ 清除所有数据 [!] |
|
| 关于 |
| └─ Bendy 记账 v1.0.0 |
|_____|
```

9. 自适应布局

9.1 AppScaffold 设计

```
class AppScaffold extends ConsumerWidget {
  final Widget child;
  const AppScaffold({required this.child});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final width = MediaQuery.sizeOf(context).width;
    final isWide = width >= 600;

    if (isWide) {
      // Desktop/Web: NavigationRail + content
      return Row(children: [
        NavigationRail(
          selectedIndex: _currentIndex(context),
          onDestinationSelected: _onNavTap,
          labelType: NavigationRailLabelType.all,
          destinations: [
            NavigationRailDestination(icon: Icon(Icons.home), label: Text('首页')),
            NavigationRailDestination(icon: Icon(Icons.account_balance), label: Text('账户')),
            NavigationRailDestination(icon: Icon(Icons.add_circle), label: Text('记账')),
            NavigationRailDestination(icon: Icon(Icons.bar_chart), label: Text('统计')),
            NavigationRailDestination(icon: Icon(Icons.settings), label: Text('设置')),
          ],
        ),
        const VerticalDivider(thickness: 1, width: 1),
        Expanded(child: child),
      ]);
    } else {
      // Mobile: BottomNavigationBar + content
      return Scaffold(
        body: child,
        bottomNavigationBar: NavigationBar(
          selectedIndex: _currentIndex(context),
          onDestinationSelected: _onNavTap,

```

```
        destinations: [/* same 5 items */],
    ),
);
}
}
}
```

9.2 响应式断点

断点	宽度	导航	卡片列数
compact	< 600	BottomNavBar	1-2
medium	600-1023	NavigationRail	2-3
expanded	>= 1024	NavigationRail (extended)	3-4

10. 预设数据

10.1 预设分类

支出分类	图标	收入分类	图标
餐饮	restaurant	工资	payments
交通	directions_car	奖金	card_giftcard
购物	shopping_cart	理财	trending_up
居住	home	兼职	work
通讯	phone_android	其他收入	star
娱乐	sports_esports		
医疗	local_hospital		
教育	school		
其他支出	inventory_2		

10.2 预设账户

账户	类型	图标
现金	asset	money
储蓄卡	asset	account_balance

账户	类型	图标
信用卡	liability	credit_card

10.3 初始化逻辑

```
@Riverpod(keepAlive: true)
class DataInit extends _$DataInit {
  @override
  Future<void> build() async {
    final db = ref.read(appDatabaseProvider);
    final existingCats = await (db.select(db.categories)).get();
    if (existingCats.isEmpty) {
      await _insertPresetCategories(db);
      await _insertPresetAccounts(db);
    }
  }
}
```

11. 项目目录结构

```
bendy-jizhang/
├── lib/
│   ├── main.dart                ← 入口 + 三端适配
│   ├── app.dart                 ← MaterialApp.router
│   ├── database/
│   │   ├── app_database.dart    ← drift Database 类
│   │   ├── app_database.g.dart  ← drift 生成代码
│   │   ├── tables/
│   │   │   ├── accounts.dart
│   │   │   ├── transactions.dart
│   │   │   └── categories.dart
│   │   └── daos/
│   │       ├── account_dao.dart
│   │       ├── transaction_dao.dart
│   │       ├── category_dao.dart
│   │       └── daos.g.dart
│   ├── repository/
│   │   ├── account_repository.dart
│   │   ├── transaction_repository.dart
│   │   └── category_repository.dart
│   ├── provider/
│   │   ├── database_provider.dart
│   │   ├── repository_provider.dart
│   │   └── data_provider.dart    ← Stream providers
```

```

├── settings_provider.dart
├── provider.g.dart          ← riverpod_generator
├── model/
│   ├── enums.dart
│   ├── ui_state.dart        ← freezed data classes
│   └── ui_state.freezed.dart
├── navigation/
│   ├── app_router.dart      ← go_router 配置
│   └── app_scaffold.dart    ← 自适应 Shell
├── screen/
│   ├── home/
│   │   └── home_screen.dart
│   ├── transaction/
│   │   ├── transaction_edit_screen.dart
│   │   └── transaction_list_screen.dart
│   ├── account/
│   │   ├── account_list_screen.dart
│   │   └── account_edit_screen.dart
│   ├── category/
│   │   ├── category_list_screen.dart
│   │   └── category_edit_screen.dart
│   ├── statistics/
│   │   └── statistics_screen.dart
│   └── settings/
│       └── settings_screen.dart
├── widget/
│   ├── number_pad_sheet.dart
│   ├── category_selection_sheet.dart
│   ├── account_selection_sheet.dart
│   ├── date_selection_sheet.dart
│   ├── color_picker_sheet.dart
│   ├── icon_picker_sheet.dart
│   └── summary_card.dart
├── util/
│   ├── amount_util.dart
│   ├── date_util.dart
│   └── color_util.dart
├── l10n/
│   ├── app_localizations.dart    ← i18n 生成
│   ├── app_en.arb
│   └── app_zh.arb
├── test/
│   ├── database_test.dart
│   ├── repository_test.dart
│   ├── util_test.dart
│   └── widget_test.dart
├── web/
│   ├── index.html
│   └── manifest.json
├── windows/                  ← Flutter 桌面生成
└── android/                  ← Flutter Android 生成

```

```
|— ios/                ← Flutter iOS 生成
|— macos/              ← Flutter macOS 生成
|— linux/              ← Flutter Linux 生成
|— pubspec.yaml
|— analysis_options.yaml
|— CLAUDE.md
|— plan.md
|— maintain.md
```

文件数统计：约 30 个 Dart 文件（原项目 90+ Kotlin 文件）

12. pubspec.yaml

```
name: bendy_jizhang
description: A cross-platform local bookkeeping app.
version: 1.0.0+1

environment:
  sdk: ^3.6.0
  flutter: ">=3.27.0"

dependencies:
  flutter:
    sdk: flutter
  flutter_localizations:
    sdk: flutter

# State
flutter_riverpod: ^2.6.1
riverpod_annotation: ^2.6.1
freezed_annotation: ^2.4.4
json_annotation: ^4.9.0

# Database
drift: ^2.22.0
drift_flutter: ^0.2.0
sqlite3_flutter_libs: ^0.5.28

# Navigation
go_router: ^14.6.2

# Charts
fl_chart: ^0.69.2

# Storage
shared_preferences: ^2.3.4
```

```
# i18n
intl: ^0.19.0

# UI
material_color_utilities: ^0.12.0

dev_dependencies:
  flutter_test:
    sdk: flutter
  build_runner: ^2.4.13
  drift_dev: ^2.22.0
  riverpod_generator: ^2.6.2
  freezed: ^2.5.7
  json_serializable: ^6.8.0
  mockito: ^5.4.5
  integration_test:
    sdk: flutter

flutter:
  generate: true
  uses-material-design: true
```

13. 三端构建与发布

13.1 Android

```
flutter build apk --release          # APK
flutter build appbundle --release    # AAB (Google Play)
```

- minSdkVersion: 21 (Flutter 默认)
- targetSdkVersion: 35
- 签名 : key.properties + upload-keystore.jks

13.2 Desktop

```
flutter build windows --release      # Windows .exe
flutter build macos --release        # macOS .app
flutter build linux --release        # Linux binary
```

- SQLite : sqlite3_flutter_libs 打包 native SQLite
- Windows : MSIX 打包可选

13.3 Web

```
flutter build web --release          # 静态文件 → build/web/
```

- SQLite : drift WASM 模式 (无需 native SQLite)
- 部署 : 任何静态托管 (GitHub Pages / Vercel / Nginx)

13.4 平台差异代码

```
// main.dart
import 'package:bendy_jizhang/app.dart';
import 'package:bendy_jizhang/database/app_database.dart';
import 'package:flutter/foundation.dart' show kIsWeb;
import 'package:flutter/material.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  // drift 会根据平台自动选择 native 或 WASM SQLite
  runApp(const BendyApp());
}
```

14. 开发计划

Phase 1 : 项目骨架 + 数据层 (v0.1.0)

- [] flutter create + pubspec.yaml 配置
- [] drift 表定义 (accounts / transactions / categories)
- [] AppDatabase + DAO + build_runner 生成
- [] 3 个 Repository
- [] Riverpod Provider (database / repository / stream)
- [] main.dart + app.dart + MaterialApp.router
- [] 主题 (Material3 dynamic color)
- [] 预设数据初始化
- [] 单元测试 : DAO + Repository

Phase 2 : 导航 + 首页 + 记账核心 (v0.2.0)

- [] go_router 路由配置 + ShellRoute
- [] AppScaffold 自适应布局 (BottomNav / Rail)
- [] HomeScreen (多周期汇总卡片 + 最近交易)

- [] TransactionEditScreen (三种类型 + 表单)
- [] TransactionListScreen (按月分组)
- [] BottomSheet 组件 (分类/账户/日期选择)
- [] NumberPadSheet (数字键盘)
- [] 桌面快捷键 (Ctrl+N)

Phase 3 : 账户 + 分类管理 (v0.3.0)

- [] AccountListScreen (资产/负债分组 + 净值)
- [] AccountEditScreen (新建/编辑)
- [] CategoryListScreen (支出/收入切换)
- [] CategoryEditScreen (二级分类支持)
- [] ColorPickerSheet + IconPickerSheet
- [] 删除确认对话框

Phase 4 : 统计分析 (v0.4.0)

- [] StatisticsScreen (日期范围 + 数据类型切换)
- [] 饼图 (fl_chart PieChart)
- [] 柱状图 (fl_chart BarChart)
- [] 趋势线 (fl_chart LineChart)
- [] 分类排行列表 + 进度条
- [] 桌面端 Tooltip + 面板布局

Phase 5 : 设置 + i18n + 发布 (v0.5.0)

- [] SettingsScreen + SettingsProvider
 - [] 暗色模式 (ThemeMode.system/light/dark)
 - [] 主题色切换
 - [] 清除数据功能
 - [] 中英文本地化 (ARB + flutter gen-l10n)
 - [] 全面单元/Widget 测试
 - [] Android 签名配置
 - [] Web 部署配置
 - [] 桌面打包配置
 - [] 应用图标
-

15. 测试策略

层	测试内容	框架
drift 表/DAO	CRUD + 查询 (NativeDatabase in-memory)	flutter_test + drift
Repository	委托 DAO 的正确性	mockito
Provider	Stream 数据转换、state 更新	flutter_riverpod + mockito
Util	AmountUtil / DateUtil / ColorUtil	flutter_test
Widget	关键 Screen 渲染 + 交互	flutter_test (WidgetTester)
Integration	端到端记账→查看→统计流程	integration_test

测试示例

```
test('AccountDao CRUD', () async {
  final db = AppDatabase.forTesting(); // NativeDatabase.memory()
  final dao = db.accountDao;

  await dao.insertOrUpdate(AccountsCompanion.insert(
    type: AccountType.asset,
    name: '测试账户',
    icon: 'account_balance',
    color: '#FF5722',
    currency: 'CNY',
  ));

  final all = await dao.watchAll().first;
  expect(all.length, 1);
  expect(all.first.name, '测试账户');

  await db.close();
});
```

16. 安全与规范

项目	说明
无硬编码密钥	纯本地无密钥
数据前缀	drift 表名默认小写复数 (accounts/transactions/categories)
版本管理	MAJOR.MINOR.PATCH, pubspec.yaml version 字段

项目	说明
Commit 规范	Conventional Commits
i18n	ARB 文件 + flutter gen-l10n
代码生成	build_runner + drift_dev + riverpod_generator + freezed
lint	analysis_options.yaml (flutter_lints + 严格模式)

17. 与原项目最终对比

维度	ezBookkeeping (Kotlin)	Bendy 记账 (Flutter)
语言	Kotlin	Dart
框架	Jetpack Compose (Android only)	Flutter (Android + Desktop + Web)
数据库	Room (Android only)	drift (跨平台 SQLite)
状态管理	Hilt + ViewModel + StateFlow	Riverpod + freezed
导航	Navigation Compose	go_router
DI	Hilt (编译期)	Riverpod (运行时，更轻)
图表	Canvas 手绘	fl_chart
i18n	strings.xml	ARB + gen-l10n
Entity 数量	14	3
文件数量	90+	~30
平台	Android only	Android + Desktop + Web
用户系统	登录/注册/OAuth/2FA	无，纯本地
网络层	Retrofit + OkHttp	无
云端同步	有	无